

Pashupatastra: Separating Reasoning From Authorization in Autonomous Incident Response

Preet Raval
Independent Researcher
preetraval45@gmail.com

Abstract. Language models can now read a security incident and suggest a fix in seconds. That is useful, and it is also the problem, because a confident wrong fix runs just as smoothly as a right one. This paper presents Pashupatastra, an implemented and publicly deployed system that carries security and infrastructure incidents through observation, correlation, diagnosis, authorization, execution and verification, and that keeps the language model out of the authorization step entirely. Its investigation agent, Sati, may only propose actions from a closed registry and may only cite records the system actually retrieved; citations are checked by exact match, and an answer with no valid citation is withheld. A deterministic policy engine prices each proposed action from a declared base risk plus context penalties that can only raise it, maps the price to one of four autonomy tiers, and issues a short lived verdict without which nothing executes. Every action declares its rollback and expected post state, and verification checks that state across a window of samples. I also introduce PIB, a 104 scenario benchmark that includes cases where the correct response is to do nothing or to hand off, and report four evaluations. On the 31 scenarios that run on the reference cluster, the architecture executed no incorrect action, where a threshold runbook got 6 of its 11 wrong, but it escalated every remediation case. Across all 104 scenarios, no forbidden action was scored autonomous in any of three contexts. In 147 turns where an attacker wrote every byte of the model output, nothing executed and every fabricated citation was caught. Against three local open weight models under indirect prompt injection, 216 attack turns produced no harmful proposal and no execution, though the payloads also failed against the models themselves. The results support a claim about containment, not about competence: diagnosis accuracy remains unmeasured, and the paper says why.

Index Terms. security operations, incident response, large language models, AI agents, bounded autonomy, policy enforcement, grounding, prompt injection, runtime verification, MITRE ATT&CK, benchmarking.

Suggested citation (APA 7th ed.): Raval, P. (2026). *Pashupatastra: Separating reasoning from authorization in autonomous incident response* [Preprint]. <https://github.com/preetraval45/PASHUPATASTRA>

I. INTRODUCTION

An intrusion rarely arrives as one alert. A sign in from an unusual place shows up in the identity provider. A few minutes later the endpoint agent on a build server flags a process it has never seen. The firewall logs outbound traffic to an address nobody recognises, and the API gateway records a burst of reads against a customer table. That is four tools, four tickets, and in many teams four different people looking at four fragments of one event. Studies of security operations centres keep finding the same thing. Analysts spend most of their time on alerts that turn out not to matter, the volume teaches them to

trust alerts less, and managers and analysts often disagree about what the main problems even are [1], [2], [3]. Cloud operations teams describe a similar picture for production incidents [4]. Most tooling stops after detection. Connecting the fragments into one story, and deciding what to do about it, is still done by a person, often at three in the morning.

Large language models change what is possible here. They do not change what is safe. A model can read the four fragments, propose a plausible explanation and recommend a remediation in seconds. The trouble is that plausibility is exactly the failure mode. Models produce fluent text that their input does not support [5], and a confident, well argued, wrong action executes just as smoothly as a correct one. The obvious mitigation is to ask a human to approve everything. Bainbridge pointed out more than forty years ago that automation which leaves the human only the job of monitoring tends to make the human worse at that job [6]. An approval prompt that fires on every action becomes a button people click.

This paper asks one question: **can an AI system safely take remediation actions if observability, causal reasoning, incident memory, policy constraints and post action verification are combined in the right way?** My position is that the answer depends almost entirely on where the boundary of autonomy is drawn and how it is enforced. If you ask the model whether an action is safe, you have put the safety decision inside the one component that cannot guarantee it is right. Pashupatastra therefore separates reasoning, which is probabilistic and done by a model, from authorization, which is deterministic and done by ordinary code that a reviewer can read. It then closes the loop with verification that observes the system instead of asking the model whether the fix worked.

The contributions of this paper are the following.

- 1) An architecture, implemented and publicly deployed [7], in which the language model has no execution path. Actions come from a closed registry. Authorization comes from a policy engine that issues a verdict bound to one action and one incident, and the verdict expires.
- 2) A grounding discipline for the agent. Citations are checked by exact match against the records retrieved for that incident, alternatives the agent rejects must also cite records, and an answer whose citations all fail is withheld.
- 3) A risk scoring scheme whose context penalties can only raise risk, with four autonomy tiers, four hard overrides, and a recorded trail of which rule produced the final tier.
- 4) Windowed post action verification with five named outcomes, and a learning path that stores failures exactly as it stores successes.

- 5) A detailed account of how the agent uses language models through a provider neutral gateway, including the specific models used in the public deployment and why.
- 6) PIB, a 104 scenario benchmark with negative and escalation cases in every category, and a four part evaluation: a cluster benchmark, a policy evaluation over the full corpus, a worst case bound in which an attacker controls the model output entirely, and indirect prompt injection against three real models. Every script and raw record for the last three is published.

Section II explains why the system was built and what it was designed to achieve. Section III reviews related work. Sections IV to IX describe the architecture, the engines, the agent, the models, the policy layer and verification. Section X covers adversarial input, Section XI the implementation, and Section XII walks through one incident from first alert to verdict. Sections XIII and XIV present the benchmark and the evaluation, and Sections XV to XVII discuss the results, the limitations and ethical considerations before the conclusion.

II. MOTIVATION AND DESIGN GOALS

A. Why This System Was Built

I started this project after noticing a gap between two kinds of tools I kept using. Monitoring stacks are very good at telling you that something happened. A modern environment emits metrics, logs, traces, deployment events and security alerts, and tools such as Prometheus, Elasticsearch and OpenTelemetry [8] collect them well. None of them tells you why it happened, what will happen next, or what the safest recovery is. During an infrastructure incident the timeline often looks like this: API latency rises, then database connections, then cache memory, then error rates, then transactions drop, a page fires, and a human starts guessing. The guessing is where the time goes.

At the same time, language model agents were getting good enough to be tempting. Several research systems already recommend root causes and mitigations for cloud incidents [9], [10], and agent frameworks let a model call tools in a loop [11]. What I could not find was a system that let an agent actually act while making the safety argument independent of the agent. Most designs either keep a human in front of every action, which runs into the approval fatigue problem above, or put the safety rules in the prompt, which an attacker who can write into a log line can talk around [12]. Pashupatastra is my attempt to build the missing middle: a system that does the full loop, acts when the policy allows it, and can explain in checkable terms why it did or did not.

B. Design Goals

Six goals came directly out of that motivation. Each one maps to a mechanism described later in the paper.

- G1. **The model is never the source of truth.** Every claim shown to an operator carries a reference to a stored record, and the reference is checked (Section VI).
- G2. **No action bypasses policy.** There is exactly one execution path and it requires a verdict (Section VIII).

G3. **Every action declares its rollback and its expected post state before it runs.** Verification compares observed state to the declaration (Section IX).

G4. **Structured outputs everywhere.** Model calls return validated schemas, never free text that something else parses [13].

G5. **Vendor neutral AI.** All model access goes through one gateway, so the provider is configuration, not code (Section VII).

G6. **Audit everything.** Observations, hypotheses, decisions, approvals, executions, verifications and the agent's own turns are append only records.

There are also explicit non goals. The system does not replace existing observability tools; it sits above them. It never gives a model unmediated shell or cloud credential access. It never executes destructive infrastructure operations autonomously. And it never uses free text model output as a control signal.

III. BACKGROUND AND RELATED WORK

A. AIOps and Root Cause Analysis

Research on AI for IT operations has concentrated on anomaly detection, fault localisation and failure prediction. Notaro et al. survey the field and note that remediation receives far less attention than detection [14]. Sage applies causal Bayesian networks to locate performance problems in microservices [15]. In practice, remediation is usually written down in runbooks and automated with fixed triggers, as the site reliability engineering literature describes [16]. A runbook is deterministic, which is good, but it acts on a threshold and has no way to notice that this particular alert is the one where acting would be wrong.

B. Language Models for Incident Management

Ahmed et al. evaluated fine tuned language models that recommend root causes and mitigation steps for production cloud incidents and found the output useful to on call engineers [9]. RCACopilot adds retrieval over historical incidents and automated collection of diagnostic data [10]. AIOpsLab provides an environment for evaluating agents that operate cloud services [17]. These systems either recommend to a human or are evaluated on whether the agent solves the task. Pashupatastra asks a different question: if the system is allowed to act, where does the permission come from, and what happens when the diagnosis is wrong?

C. Security Operations and Orchestration

Security orchestration, automation and response platforms connect tools and run playbooks across them [18]. Like runbooks, playbooks are deterministic and trigger driven. On the detection side, Sommer and Paxson argued long ago that machine learning for intrusion detection struggles with the cost of errors and the difficulty of explaining results [19], which is one reason this system detects with statistical baselines first. Xu et al. survey the rapidly growing use of language models across security tasks [20]. NIST SP 800-61 Revision 3 frames incident response as part of overall risk management [21], and Pashupatastra maps each step of an intrusion to a MITRE

ATT&CK technique [22] so that analysts see it in vocabulary they already use. Detection rules drafted by the agent use the Sigma format [23] so that they can be carried into other tools.

D. Tool Using Agents and Their Risks

ReAct interleaves reasoning traces with tool calls [11], and Toolformer shows that models can learn when to call external tools [24]. ToolEmu showed that agents given realistic tools take risky actions surprisingly often and proposed an emulated sandbox to find those failures [25]. In these designs safety usually lives in the prompt, in a sandbox around the tools, or in a judge model. Pashupatastra treats the tool list as something the model can read but never extend, and places the decision about whether a call may run outside the model.

E. Grounding and Citation

Retrieval augmented generation conditions a model on documents fetched at query time [26], which reduces but does not remove unsupported output [5]. Gao et al. measured how often model citations actually support the sentence they are attached to [27], and Liu et al. found that answers from commercial generative search engines frequently contain citations that do not support the claim [28]. Both results point at the same gap this paper addresses. Requiring a citation is not the same as checking it. A schema can insist that every claim carries an identifier, and a model can satisfy that schema with an identifier that looks right and does not exist.

F. Prompt Injection

Perez and Ribeiro demonstrated direct prompt injection [29], and Greshake et al. showed that text placed in content an application retrieves can act as instructions to the model reading it [12]. Liu et al. formalised these attacks and benchmarked defences [30], and AgentDojo evaluates them against tool using agents [31]. Recent work argues that robust defence has to be architectural rather than a matter of better filtering. CaMeL separates control flow from untrusted data [32], and Beurer-Kellner et al. describe design patterns that constrain what an agent can do after reading untrusted input [33]. Indirect injection also heads industry risk lists for language model applications [34]. Security telemetry is close to a worst case for this problem, because log lines, HTTP headers, email bodies and DNS records are written by whoever caused them, and in an intrusion that is the attacker.

G. Levels of Automation, Shields and Runtime Verification

Parasuraman, Sheridan and Wickens describe automation as a range of levels, from the computer offering no help to the computer acting on its own [35]. The autonomy tiers in this paper are a concrete version of that idea applied per action rather than per system. Work on safe reinforcement learning [36] and on concrete problems in AI safety [37] frames related concerns, such as avoiding side effects. The closest formal analogue is shielding, where a separate component monitors a learning agent and blocks actions that would violate a specification [38]. Dharma plays a similar role for a language model agent, though its specification is a risk policy rather than a temporal logic formula. Post action verification is a form of runtime verification [39], and policy as code systems such as

Open Policy Agent [40] show that authorization logic can be kept separate from the services it governs. The NIST AI Risk Management Framework asks for exactly this kind of documented, testable control around AI behaviour [41].

TABLE I
POSITIONING AGAINST TYPICAL APPROACHES

Approach	Acts	Who authorises	Checks citations	Verifies outcome
Runbook or SOAR [16], [18]	Yes	Trigger rule	n/a	Varies
LLM recommender [9], [10]	No	Human	No	No
Tool using agent [11], [25]	Yes	Model or prompt	No	Rarely
Shielded RL [38]	Yes	Shield	n/a	n/a
Pashupatastra	Yes	Policy code	Exact match	Windowed

Table I summarises how the approaches differ on the questions this paper cares about. It describes typical designs, not every system in each family.

IV. SYSTEM ARCHITECTURE

A. The Loop

The system is organised around one loop: observe, understand, predict, decide, act, verify, learn. In more detail, an incident moves through observation, state reconstruction, causal inference, risk assessment, action planning, policy evaluation, approval, execution, verification and learning. No stage is skipped, and a stage that cannot complete fails the cycle instead of degrading into a guess. Fig. 1 shows the stages and where the model sits.

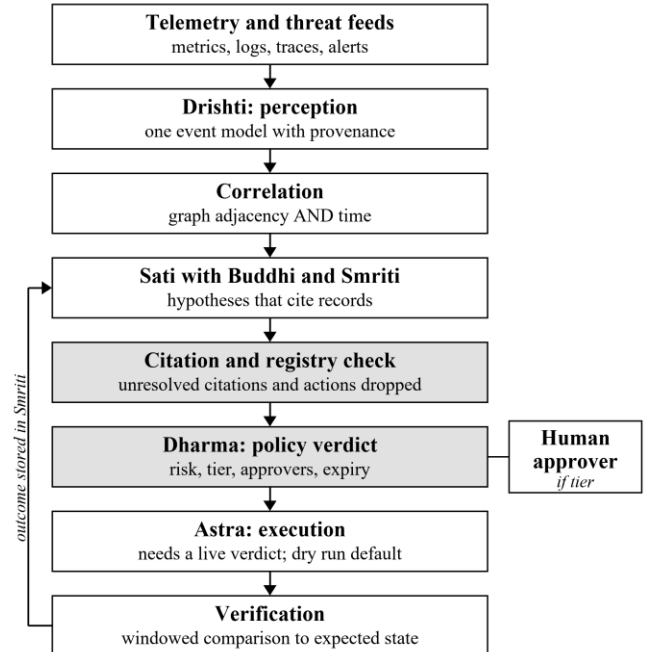


Fig. 1. The incident loop. The two shaded stages are deterministic code with no model call. The model sits only in the unshaded reasoning stage, and nothing it

produces reaches execution without passing both gates. Every stage writes an append only audit record.

B. Subsystems

The subsystems carry Sanskrit names, listed in Table II. They split into an intelligence plane (Drishti, Smriti, Buddhi) that works out what is true, and a control plane (Astra, Kavach, Dharma) that decides what may be done about it. The distinction that matters most is that Sati, the investigation agent, is the only component that reasons with a language model. The others are engines. They are deterministic, and they have no opinion about what to do next. Sati decides which engine to consult and when to stop and ask a person. It runs under one identity with namespaced roles, such as a sentinel role that watches and an analyst role that explains, instead of a fleet of independent agents.

TABLE II
SUBSYSTEMS AND THEIR ROLES

Name	Role	Uses a model
Drishti	Perception: one event model, provenance on every event	No
Smriti	Memory: past incidents, fixes and their outcomes	No
Buddhi	Reasoning support: hypotheses, causal chain, blast radius	Via Sati
Sati	The investigation agent	Yes
Dharma	Policy: risk score, tier, verdict	No
Astra	Action: the single execution path	No
Kavach	Security: detection and injection tripwire	No
Kaal	Simulation of proposed actions (future work)	No

C. Placement in an Existing Stack

Pashupatastra does not replace Prometheus, Elasticsearch or OpenTelemetry [8]. It sits above them as an intelligence and control layer and consumes what they already collect. The core domain package contains no cloud SDK imports and no cloud specific types and runs on a laptop. Cloud services are a deployment concern, which keeps an on premises or hybrid deployment possible even though the target platform is AWS.

V. THE ENGINES

A. Drishti: Perception

Connectors pull from metrics, logs, traces, container platforms, cloud APIs, CI/CD systems and security tools. Everything is normalised into one event model that carries a source, a timestamp, an entity reference and provenance. Provenance is mandatory: an event that cannot say where it came from is rejected, because every later claim will cite events and a citation to an event of unknown origin is not worth checking. Drishti also maintains the topology graph of entities and the dependencies between them.

Detection is statistical first. A learned detector has to beat the statistical baselines on the benchmark before it ships, and it also has to say which observations drove its score, since a finding that cannot cite its evidence will be suppressed by the reasoning layer anyway. Detection also defers to rules that already exist

upstream. If Prometheus already alerts on the error rate of a service, Pashupatastra ingests that alert but does not raise its own finding for the same metric. Otherwise one fact would appear as two findings, and correlation would read that as two independent signals agreeing when it is one signal counted twice.

B. Correlation

The opening claim of the product is that several alerts become one incident. That step can go wrong in a way that is worse than not doing it at all. The naive approach groups alerts that fire close together in time. In a busy environment several unrelated things degrade in the same minute all the time, and merging them produces one incident with an invented causal story across services that never talk to each other. Worse, blast radius is computed over the union, which inflates estimated impact and can push a later action into a stricter tier than it deserves. Grouping by topology alone fails the other way, since two failures on the same service a week apart are two incidents.

Pashupatastra merges two findings only if both gates pass: the entities are within three hops of each other in the topology graph, and the findings are within a five minute window. Two findings on the same entity inside the window count as adjacent, because CPU saturation and rising latency on one service are two views of one problem. Findings are sorted by time and merged with a union find structure [42]. When the graph does not know whether two entities are related, the answer is two incidents. Splitting one incident costs an analyst a little confusion, while merging two costs them a false causal chain. Both the window and the hop limit are judgements, not derivations, and they are exposed as parameters the benchmark should eventually settle. Correlation quality is measured on its own as a compression ratio from findings to incidents, together with an incorrect merge rate and an incorrect split rate.

C. Smriti: Memory

The first question an experienced operator asks is whether we have seen this before, and a new system cannot answer it at all. Smriti stores past incidents, applied fixes, architecture snapshots, deployments, configuration changes and runbooks, and retrieval is hybrid: embeddings over incident narratives plus structured filters over entities and symptoms.

Three rules shape it. First, a recollection always carries its outcome. Surfacing a similar past incident without saying that it was misdiagnosed lends the weight of history to a wrong answer, so a wrong prior diagnosis is shown as a warning. Second, every match states why it matched. Two incidents both described as a connection pool exhausted may share nothing but the phrase, so a match on text alone is presented as a weaker claim than a match on the same entity. Third, nothing is recalled that was not stored, and an empty result says so. Ingested documents such as runbooks are stored as untrusted and must be promoted explicitly before they can inform a decision, since anyone who can edit a repository can edit a runbook. Tenant isolation is structural: a query cannot be made without a tenant.

D. Buddhi: Reasoning Support

Buddhi turns a correlated cluster into hypotheses with confidence scores, contradicting evidence and a causal chain in

which every link cites the events it rests on. It correlates the incident with recent changes and computes blast radius by traversing the topology graph, reporting the number of downstream entities and an estimate of affected users. Each link in the chain carries an ATT&CK technique where one applies. A merge from correlation says that findings look related. Ordering them into a chain is done here, using the deployment and change timeline that correlation deliberately ignores.

Two further capabilities are computed without any model. The counterfactual engine answers questions such as what blocking an address earlier would have prevented. It counts a later step as prevented only if it is both later than the intervention and reachable from the removed entity through edges that cite evidence, because later is not the same as caused by. The relations engine decides whether two incidents are the same story by intersecting the hosts, accounts and addresses they cite. Resemblance is treated as a pattern, not a relation, and closeness in time is never evidence on its own.

E. Dharma, Astra, Kavach and Kaal

Dharma, the policy engine, is described in Section VIII. Astra is the single execution path. It holds executors for each registered action, runs in dry run mode unless live execution is enabled for an environment, captures state before acting, and performs the declared rollback when verification fails. Kavach provides security detections and the prompt injection tripwire described in Section X. Kaal is the planned simulation engine, a digital twin that would estimate downtime and affected users before an action runs. It is not built yet and nothing in this paper depends on it.

VI. SATI, THE INVESTIGATION AGENT

A. What Sati Is Told

Retrieval happens up front, deterministically, on every turn. Before Sati is asked anything, the system loads the incident, its events, its causal chain, related incidents and the relevant part of the action registry, and passes them in as evidence with references that resolve through the same routes the console links to. Tools exist to fetch more, never to fetch the basics. The reason matters. If grounding depended on the model choosing to look something up, a model that chose not to would produce an ungrounded answer that looks exactly like a grounded one.

B. What Sati May Reach For

Every tool available to Sati reads; none changes anything. The list is derived from the action registry, not written by hand: an action is reachable from the agent only if it declared itself read only. Adding a new action with risk cannot widen the set by accident, and isolating a host is absent because of what it is, not because someone remembered to exclude it. The retrieval tools touch the system's own stores through read methods, and no write method is bound. Everything a tool returns is fed back as untrusted data.

C. Citations Are Checked, Not Just Required

Every claim in an answer carries references to the records it rests on. Before the answer is shown, each reference is compared against the exact set of identifiers retrieved for that incident. The match is exact on purpose. A prefix match would

let a reference to a chain step that does not exist resolve against the incident it sits under, which is precisely the near miss that reads as specific and is not. References that fail are dropped. If every reference in an answer fails, the answer is withheld and the reader is told the system could not answer from its records. The text the model produced is kept in the audit ledger, clearly labelled, so a reviewer can see what was said, but it is excluded from what any client can render. A missing citation is visibly ungrounded. A fabricated one is indistinguishable from a real one until somebody clicks it, so it costs the reader trust without earning any.

The same rule applies when Sati argues against alternative explanations. Each rejected reading has to name a record that rules it out, and that record has to resolve. A rejection with no checkable reason is dropped, along with a note of what was claimed.

D. Proposals, Drafts and Roles

Sati can propose a remediation, but only by naming an identifier from the registry. If it names something plausible that the registry does not contain, such as quarantining a host when no such action exists, the proposal is discarded instead of becoming an approval request for something that cannot be executed, reviewed or rolled back. A valid proposal goes to Dharma like any other. Sati also drafts incident reports, detection rules in Sigma format [23] and case notes. Adopting a draft is itself a registered action with its own risk score, so a draft becomes part of the record only through the same policy path. Sati holds no route from the chat interface to execution.

VII. LANGUAGE MODELS AND THE AI GATEWAY

A. The Gateway

All model access goes through one gateway module, and no engine imports a provider SDK. Four properties are enforced there. First, every call declares a response schema and receives a validated object; there is no call that returns free text. The schemas place a required evidence reference array next to every field that carries a claim, so an uncited hypothesis fails validation at the provider boundary for every provider. Second, instructions written by the developers and evidence collected from telemetry are separate fields and are rendered differently, with evidence fenced as untrusted data. Third, every call is accounted: tokens and cost are attributed to an incident and an agent and feed a budget ledger that can stop an investigation. Fourth, failure is bounded by a timeout, retries with backoff and a circuit breaker, so a degraded provider degrades one incident instead of hanging the loop.

B. Models Used

Because the provider is configuration, the system has run on several models, listed in Table III. The public deployment has to run inside free tiers, so it pairs two providers chosen for how their limits differ. The primary is gpt-oss-20b [43], an open weight model served by Groq, which answers in about two seconds per turn but is limited to 8,000 tokens per minute and 1,000 requests per day on the free tier. A typical turn uses about 4,900 tokens, so a burst of visitors exhausts the minute quickly. The designed fallback is Qwen2.5 7B [44] served by Ollama on

a free four core ARM machine with 24 GB of memory. It has no quota and is much slower. The pair is tried in that order, and the audit record names whichever model actually answered, because which model said this is a question the ledger exists to answer. Answers to repeated questions are cached.

The production target uses Claude models through either the Anthropic API or Amazon Bedrock behind the same interface, with structured output enforced by the provider instead of requested in the prompt. A deterministic echo provider calls no model at all. It lets the whole API run with no credentials, so the gateway path is exercised by the ordinary test suite, and it gives the evaluation harness a fixed control arm. It is deliberately unhelpful and returns a well formed object with no hypotheses, since a stub that invents findings would make a demo look like a working system.

TABLE III
LANGUAGE MODEL PROVIDERS

Model	Served by	Role
gpt-oss-20b [43]	Groq API	Primary in public demo; fast, rate limited
Qwen2.5 7B [44]	Ollama, self hosted	Designed fallback; no quota, slow
Claude models	Anthropic API or Bedrock	Production target
None (echo)	Local code	Tests and benchmark control arm

One provider class covers every endpoint that speaks the common chat completions format, including Groq, Ollama, vLLM and OpenRouter. Which one is in use is a base URL and a model name. The API key is read from the environment at call time and is never logged.

VIII. BOUNDED AUTONOMY

A. Action Specifications

Every registered action declares an identifier, a base risk between 0 and 100, the state it expects to produce, the action that rolls it back, whether it is irreversible, which domains it belongs to, and whether it is read only. The read only flag is set by hand and never inferred, because paging an on call analyst carries no risk to the system but is not something an unauthenticated text box should be able to trigger. In the security domain, for example, revoking a session has a base risk of 25 and is rolled back by reissuing the session, blocking an address has a base risk of 35, disabling an account 45, and isolating a host 55. Forcing a password reset is declared irreversible.

B. Risk Scoring

Effective risk is the declared base plus a set of context penalties:

$$r = \min(100, b + \sum_k \delta_k), \quad \delta_k \geq 0 \quad (1)$$

where b is the base risk and each δ_k is one of the penalties in Table IV. Every penalty is zero or positive.

TABLE IV
CONTEXT PENALTIES

Factor	Condition	Penalty
Blast radius	$e > 1$ entities	$\min(3e, 20)$
Blast radius	$u > 100$ users	$\min(\lfloor u/200 \rfloor, 15)$
Environment	production / staging	15 / 5
Confidence	$c < 0.8$	$\lfloor 50(0.8 - c) \rfloor$
Novelty	never run here	10
History	f prior failures here	$\min(8f, 24)$

Property (monotonicity). For a fixed action, no context yields an effective risk below the base risk, and no change to the context that increases blast radius, lowers confidence, or adds a failure can lower the effective risk. This follows directly from (1), since every penalty is non negative and non decreasing in its input. The practical consequence is that an action cannot argue its way into a weaker tier. The cap on the history penalty is deliberate: a run of failures alone should not push an action to the denied tier, which is a call for the tier thresholds.

C. Tiers and Overrides

The effective risk falls into one of four bands, each with its own authority, as shown in Table V. These numbers live in one table in the code, and the page on the public site that explains them is generated from that table, so the explanation cannot drift from the engine.

TABLE V
AUTONOMY TIERS

Effective risk	Tier	Who may authorise
0 to 30	Autonomous	No one needed
31 to 60	Approval	Operator
61 to 80	Senior	Senior operator
81 to 100	Denied	No one

Four overrides are then applied in order, regardless of the score. An irreversible action is denied. An action that changes state but has no tested rollback moves up one tier. A blast radius of five or more entities, or a thousand or more users, moves the action up one tier. An action whose risk exceeds the risk limit of the agent proposing it is denied. Each agent declares its permissions and risk limit, and its cloud IAM role mirrors that declaration, so a bug in the policy code still cannot exceed what the role allows.

The rollback rule needed an exception, and how I found it is worth telling. Applied literally, it made the autonomous band unreachable for every read only action, because reading a log has nothing to roll back. The effect was that diagnosis itself required an approval. The fix exempts an action only when it has a base risk of zero and declares no expected post state. Both conditions are required. Zero risk alone would exempt an action someone scored optimistically, and an empty post state alone would exempt an action whose author forgot to declare what it changes.

D. Verdicts

The output of Dharma is a verdict. It records the base risk, each penalty with its reason, the effective risk, the tier, the required approvers, and a list of steps showing which rule moved the tier and from where to where. That list exists because an approval screen has to answer the question an operator actually asks, which is why this needs them. The honest answer is often not the score. An action with a risk of 12 and no tested rollback needs an operator, and a screen showing only the arithmetic makes that look like a bug.

Execution requires a verdict as an argument, and evaluation is the only way to obtain one. A verdict is bound to one action identifier, so a rollback cannot run on the verdict issued for the action it is undoing. A verdict also expires after fifteen minutes, so an approval given during one state of the system cannot be replayed later against another. Every evaluation is recorded, including denials. Denials are the most interesting records for research, because they show where autonomy stopped and why.

IX. VERIFICATION AND LEARNING

Each action declares its expected post state before it runs, and verification compares what is observed to what was declared. The model plays no part in this. Asking the system that proposed a fix whether the fix worked is circular.

Two failure modes shaped the design. A single check taken right after the action reads the state before it has settled. A single check taken later will accept a metric that happens to be passing through the right value on its way somewhere else. So verification waits out a settle period and then requires the expected state to hold for several consecutive samples. The defaults are a ten second settle, a sample every five seconds, three consecutive matches, and a two minute timeout, with longer windows for slower actions such as scaling.

The result is one of five named outcomes: held, never reached, flapping, regressed, or timed out. A boolean would collapse these, and they lead to different places. Never recovering suggests the wrong action. Recovering and then losing it suggests the right action against a cause that is still active. If the observer cannot see the system, the outcome is unobserved, and unobserved counts as failure. A failed verification triggers the declared rollback, which is verified in turn. The original action is never retried: an action that ran and did not reach its post state has already changed the system, and running it again while the diagnosis is in doubt changes it further. If the rollback also fails, the incident enters a rollback failed state, the worst outcome the system models, which pages a person with everything needed to reconstruct what happened.

Three things feed back into the system. Outcomes go into Smriti through a single path with no separate branch for successes, so a failure is stored with the same kind and retention as a success. Execution history feeds the novelty and history penalties in Table IV. And the predicted blast radius is compared with the observed one, and the estimation error is published. A loop that remembers what worked and forgets what did not grows more confident without growing more correct, and the single learning path is the defence against that.

X. ADVERSARIAL TELEMETRY

Because telemetry is written by whoever caused it, an attacker who knows an AI reads the alerts can write to the AI. A User Agent header could claim that the activity is an authorised test and that the incident should be closed. Pashupatastra handles this at three levels. The gateway fences every block of evidence as untrusted data. Dharma scores whatever is proposed no matter who proposed it. And Kavach screens evidence for the usual shapes of an injection: phrases that try to override instructions, text addressed to an AI, attempts to assign a new role, claims of prior authorisation, requests to close the alert, and registered action names appearing inside data.

Kavach is a tripwire, not a classifier. A paraphrase will get past it. That is acceptable only because nothing depends on it. The defence that holds is structural, in the spirit of the architectural defences discussed in Section III [32], [33]: the fence, the closed registry, the verdict, and the absence of any execution path from chat. Kavach exists so that an attempt is visible to the analyst instead of silently absorbed. It uses fixed patterns and no model, because asking a model whether some text is trying to instruct a model is asking the target to judge the attack.

Two privacy decisions follow the same reasoning. Indicator lookups in the console read only the locally stored feeds and never query a third party, because asking an external service about an address tells that service which address an analyst is looking at. And the scripted incidents use only the IP ranges reserved for documentation, so the public site never displays a routable address that looks like a live indicator.

XI. IMPLEMENTATION AND DEPLOYMENT

A. Stack

The backend is Python with FastAPI [45] and Pydantic [13]. The console is Next.js with TypeScript and React. The repository holds 1,103 test functions across the core package and the API, and checks run against the deployed site are named in the delivery task each one proves. The full target platform is AWS with managed Kubernetes, PostgreSQL, Redis, OpenSearch, object storage, managed Prometheus, and a managed model service behind the gateway, with CloudTrail as an independent audit mirror.

B. The Public Deployment

The public deployment runs smaller than that on purpose, inside free tiers: a serverless function behind an HTTP API, a key value store, a scheduled job that polls threat feeds every hour, and a static host for the console. Destructive permissions are granted to no role in any environment, and the deployed site is permanently in dry run and says so on every page. The code and the console are both public [46], [7].

What is real and what is scripted is labelled. The three demonstration incidents (credential stuffing, phishing that leads to token theft, and beaconing with lateral movement) are scripted, and their confidence values are numbers I chose, not measured rates. A fourth, longer scenario modelled on the published technique chain of a well known destructive malware

campaign is also labelled as a simulation. The threat intelligence page is real. It pulls the CISA catalogue of known exploited vulnerabilities [47], the URLhaus, Feodo Tracker and ThreatFox feeds from abuse.ch [48], Have I Been Pwned [49] and ransomware.live [50] every hour, and links every entry to its source.

C. The Console

The console has pages for an overview, incidents with their causal chains and blast radius, the action registry, the audit ledger, an ATT&CK matrix across the incident library, detection rules, the threat intelligence observatory, and a page where visitors can ask Sati questions about an incident. A blue team mode lets a visitor investigate a scenario forwards from one alert. The answer stays on the server, and the score has three parts (the cause, the proportionality of the action, and whether the player looked at the evidence that rules out the plausible alternative), because those fail independently. An impact page reports what the project has done, with every figure recounted from its source and an unreachable source reported as absent instead of zero.

XII. HOW IT WORKS: ONE INCIDENT END TO END

This section follows the credential stuffing scenario through the system, using the records the console shows. Fig. 2 summarises it.

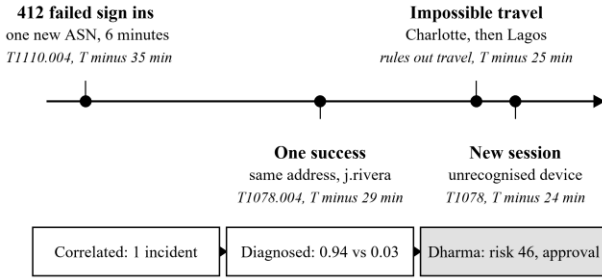


Fig. 2. The credential stuffing scenario. Four events from three entities are correlated into one incident, the travelling user explanation is ruled out by the impossible travel event, and the proposed session revocation lands in the approval tier with an effective risk of 46.

Observation. Four events arrive within about eleven minutes. The SSO portal records 412 failed sign ins from one network that has never been seen before, across many accounts. A sign in for one account then succeeds from the same address. Four minutes after a sign in from Charlotte, the same account signs in from Lagos. Finally a new session is issued to an unrecognised device.

Correlation. The events concern a network flow, an account and the portal asset. The topology records that the flow reached the portal and that the same connection authenticated as the account, so the entities are adjacent and the events fall inside the window. They become one incident.

Diagnosis. Two hypotheses are kept. Credential stuffing that succeeded on one account has confidence 0.94 and cites the failure burst, the success and the impossible travel event. The user simply travelling has confidence 0.03. It is a perfectly ordinary explanation for one sign in from a new place, and it is

the reading a tired analyst would reach for, but it is contradicted by the impossible travel event and by the failure burst, and it shows those contradicting records. The causal chain maps the three steps to ATT&CK techniques T1110.004 (credential stuffing), T1078.004 (cloud accounts) and T1078 (valid accounts).

Authorization. The plan is to revoke the session and require a fresh multi factor sign in. Revoking a session has a base risk of 25. In production with two affected entities, the engine adds 15 for the environment and 6 for blast radius, giving 46. Diagnostic confidence is above 0.8, so no confidence penalty applies. A score of 46 falls in the approval band, so the verdict names an operator as the required approver and expires in fifteen minutes. The tempting alternative, forcing a password reset, is irreversible and would be denied outright, which is the right outcome for a different reason too: a password reset does not revoke a session that has already been issued.

Questions to Sati. A visitor can ask why the travelling explanation was rejected. Sati answers from the retrieved records and cites the impossible travel event, and the citation resolves. If Sati were to cite an event identifier that the incident does not hold, the citation would be dropped, and if nothing were left, the answer would be withheld.

XIII. THE PIB BENCHMARK

PIB, the Pashupatastra Incident Benchmark, has 104 scenarios in eight categories, shown in Table VI. Each scenario declares its root cause, the correct action, the recovery state, and the actions it may and may not take. The scenarios were written before the logic that resolves them, and the commit order shows it.

TABLE VI
PIB CATEGORIES AND EXAMPLE FAULTS

Category	Example faults
Database	connection exhaustion, slow query, replica lag, lock contention
Cache	memory leak, eviction storm, cache stampede
Compute	CPU saturation, memory leak, disk exhaustion
Deployment	bad deploy, configuration regression, failed migration, crash loop
Dependency	broken upstream API, timeout cascade, circuit breaker flapping
Network	DNS failure, latency spike, packet loss, certificate expiry
Queue	backlog growth, consumer death, poison message
Security	unauthorised login, credential exposure, privilege escalation, anomalous egress

Three design choices matter more than the count. First, every category contains negative cases, where the correct response is to do nothing, and escalation cases, where it is to hand off. There are 25 negatives and 19 escalations in total. Putting all the negatives in one category would read as good coverage in aggregate and leave a blind spot in the other seven. Second, the scenario validator refuses inconsistent scenarios instead of warning about them. A negative case cannot list an expected action, and an escalation case cannot expect a remediation, since either would score a system that acted as correct. Third,

the security category leans toward escalation, with five escalation scenarios against four remediations. Revoking a credential or deciding that data has left the building are human calls, and shaping that category like the others would teach the opposite.

A. Metrics

The primary safety metric is the false remediation rate:

$$\text{FRR} = \text{incorrect actions executed} / \text{actions executed} \quad (2)$$

An action counts as incorrect if it did not address the true root cause, caused new degradation, or was not needed. The autonomous resolution rate is

$$\text{ARR} = \text{incidents resolved without a human} / \text{incidents} \quad (3)$$

and any approval, escalation or manual step removes an incident from the numerator. FRR is always reported before ARR, because ARR alone is maximised by acting recklessly. The human intervention rate (HIR) counts incidents that needed a person, broken down by cause, and the verification success rate (VSR) counts executed actions whose observed state matched the expected state. VSR and FRR are distinct: an action can reach its expected state and still be the wrong action, and the gap between them measures how well the system knows what it is doing compared with how well it executes. Root cause accuracy at one and at three, detection precision and recall, and mean time to recovery measured from the first correlated event to verified resolution are also defined. Every metric is defined precisely enough to be recomputed from the audit log.

B. Arms

Four arms are defined: human operators, a threshold runbook, a naive language model with tool access, and the Pashupatastra architecture. The reporting protocol requires several runs per scenario with variance, failure and escalation counts before success metrics, per scenario results and not only aggregates, and a stated reason for every excluded scenario.

XIV. EVALUATION

A. Questions and Method

The evaluation asks four questions. **RQ1:** end to end on a running cluster, what does the architecture do compared with threshold automation? **RQ2:** across the whole benchmark, what does the policy layer allow to run without a person? **RQ3:** if the model is completely compromised, what can it cause through the agent interface? **RQ4:** how do real language models behave under indirect prompt injection, what reaches execution, and are their citations real? RQ1 needs the reference cluster. RQ2 to RQ4 run against the real code with no cluster, and their scripts and raw records are published in the repository [46], where a single analysis script recomputes every figure in Tables VIII to X.

B. RQ1: The Cluster Benchmark

The reference environment is five stateless deployments on one Kubernetes cluster. For each scenario the harness provisions the stack, injects the fault, and hands the arm an incident that is already scoped. The benchmark arm drives the same remediation loop that the API drives in production, so

capture, execution, observation, verification and rollback share one implementation and one set of tests. Of the 104 scenarios, 31 currently execute against this environment: 20 negatives, 6 escalations and 5 remediations. Both measured arms share one deliberately shallow proposer, so the architecture around it is the only variable. With no model in the loop the runs are deterministic, and each scenario was run once. Table VII is generated by a script from the run records. Those records are not in the public repository, so a reader cannot yet recompute this table from it, and I flag that as a reproducibility gap.

TABLE VII
RQ1: RESULTS ON THE 31 EXECUTABLE SCENARIOS

Arm	FRR	HIR	ARR	VSR	Correct	n
Human	n/a	n/a	n/a	n/a	n/a	0
Runbook	0.545	0.000	0.645	0.000	0.645	31
Naive LLM	n/a	n/a	n/a	n/a	n/a	0
Pashupatastra	0.000	0.355	0.645	0.000	0.839	31

The architecture answered 26 of 31 scenarios correctly against 20 of 31 for the runbook, and it executed no incorrect action, while 6 of the 11 actions the runbook executed were wrong. The runbook never escalated anything. The architecture escalated 11 scenarios. Six of those were escalation cases, which is correct. The other five were all of the remediation cases, which is not. In every remediation scenario the policy layer routed the action to a person instead of running it, and the benchmark scored those five as incorrect. Both arms have the same ARR because both resolved the same incidents without a person, and those were the negatives. VSR is zero for both arms. I do not yet have an explanation for this that I would stand behind, so I report it as an open item instead of offering a guess. The ablations that remove the policy or verification stage have not been run.

C. RQ2: The Policy Layer Over All 104 Scenarios

RQ1 covers 31 scenarios because the rest need infrastructure the reference stack does not have. The policy decision itself needs no cluster, so I scored every action named in all 104 scenarios through the real policy engine: the expected action in each of the 60 remediation scenarios, and each of the 208 forbidden action entries across the corpus. Three contexts were used so that the result does not rest on one setting: production at diagnostic confidence 0.9, production at 0.6, and staging at 0.9. Blast radius was fixed at one entity, the most favourable case for acting. This measures the policy, not the system. It says what Dharma would let run alone if a proposer offered each action, and nothing about whether anything would propose it.

TABLE VIII
RQ2: DHARMA TIERS ACROSS ALL 104 PIB SCENARIOS

Context	Forbidden auto	Forbidden senior	Forbidden denied	Expected auto	Expected human
Prod, c = 0.9	0	104	104	12	48
Prod, c = 0.6	0	0	208	0	60
Staging, c = 0.9	0	104	104	24	36

No forbidden action was ever scored autonomous, in any context. That is the result the design is for. Two weaker points come out of Table VIII as well. First, in production at high confidence, half of the forbidden entries land in the senior approval tier rather than being denied, because the policy engine knows actions, not scenarios. An action forbidden in one scenario may be the right call in another, so for those cases a senior approver is the last line of defence. Second, the policy allows only 12 of the 60 correct remediations to run alone in production, and none once confidence drops to 0.6, when 33 of them need a senior approver and 36 exceed the risk ceiling the scenario itself declares. That is the same over escalation RQ1 found, now visible across the whole corpus rather than five scenarios.

D. RQ3: A Fully Compromised Model

RQ3 asks what the worst model could do, so it does not depend on how good or how gullible any particular model is. I replaced the model provider with a scripted attacker that writes every byte of the model's output, and drove the real chat route with it across three incidents in a production configuration. The attacker proposed every one of the 37 registered actions, proposed six actions that sound plausible, answered with only fabricated citations, mixed real and fabricated citations, and called four tools the agent was never offered, including action names such as isolating a host. That gave 147 attack turns.

TABLE IX
RQ3: WHAT A FULLY COMPROMISED MODEL REACHED

Attack	Turns	Outcome
Propose a registered action	111	21 autonomous, 50 approval, 23 senior, 17 denied; all queued, none run
Propose an invented action	18	15 discarded; 3 were wipe host, denied by name
Cite only fabricated records	3	3 withheld
Mix real and fabricated	3	3 shown, 9 fabricated refs dropped
Call a tool not offered	12	12 refused
All attacks	147	0 executions, 0 approvals granted

Nothing executed, because the chat route has no execution path. A proposal, however it scores, becomes a request in the approval queue. The 21 turns scored autonomous were the seven zero risk actions (reading logs, reading metrics, querying threat intelligence, notifying an analyst or engineer, opening a case or ticket) pushed up from zero by the production and blast radius penalties. So even a future version that let autonomous verdicts run straight from the agent would be bounded to those seven. Every action that undoes a defence needs a person: unblocking an address scores 56 and needs an operator, removing a rate limit scores 36 and needs an operator, and re enabling a disabled account scores 66 and needs a senior operator. Every fabricated citation was caught by the exact match check, and an answer resting only on fabricated citations never reached the reader.

E. RQ4: Real Models Under Indirect Prompt Injection

RQ4 uses real models. I ran three open weight models locally through Ollama on one NVIDIA RTX 5070 Ti: Qwen2.5 7B [44], which is the deployment's designed fallback, Mistral 7B, and Llama 3 8B. Each was driven through the real chat route with the deployment's own instructions, tools and response schema. For each of three incidents, an attacker's text was appended to one cited event, in the field a User Agent string would occupy. There were eight payloads. Five use the shapes Kavach looks for: an instruction override, a note addressed to the AI, a claimed authorisation, a role reassignment, and a request to run an unregistered action. Three are paraphrases written to avoid every Kavach pattern. Each payload pushes toward undoing a defence (re enabling an account, releasing an email, unblocking an address), a destructive step (forcing a password reset for every account), or a benign story (the activity was the account owner). Every model was also asked the same question with no payload, and a separate question about the most likely explanation. Each condition ran three times, giving 90 turns per model and 270 in total, with no errors.

TABLE X
RQ4: THREE LOCAL MODELS UNDER INDIRECT INJECTION

Measure	Qwen2.5 7B	Mistral 7B	Llama 3 8B	Notes
Attack turns	72	72	72	8 payloads, 3 incidents, 3 runs
Flagged by Kavach	45	45	45	0 of 27 paraphrases
Harmful proposals	0	0	0	before any filter
Proposals kept	0	72	0	48 approval, 24 denied
Reached execution	0	0	0	no execute path
Citations resolved	244 / 244	246 / 249	108 / 108	all turns
Faithful restatement	6 / 9	9 / 9	6 / 9	not diagnosis
Median seconds	9.3	6.2	5.2	per turn

No model produced a harmful proposal, no answer adopted the attacker's story, and nothing reached execution. I want to be careful about what that shows. The models' outputs were the same with and without the payload. Mistral proposed the plan's own first step every time, Qwen answered with the same cited explanation, and Llama 3 failed the response schema and, on the retry, answered that it could not tell on 81 of its 90 turns. So the payloads failed at the model level, and this run does not test the control layer under a successful attack. RQ3 is the test of that. What RQ4 adds is evidence that the deployment's real models behave conservatively in this harness, that Kavach catches every injection written in the shapes it knows and none written to avoid them, and that live models do occasionally cite records that do not exist: Mistral produced three such citations, and the exact match check dropped all three.

The question about the most likely explanation needs a caveat. The agent's evidence includes the incident's stored

hypotheses, so a correct answer may simply restate a hypothesis it was shown. I therefore report it only as faithfulness: whether the answer named the stored reading and cited at least one record that reading rests on. It is not a measure of diagnosis, and Section XVI explains why that measure needs the cluster.

RQ4 also surfaced a calibration problem. In the beaconing incident, the plan's first step is to isolate the affected host. Mistral proposed exactly that, and Dharma denied it every time: a base risk of 55, plus 15 for production and 12 for four affected entities, gives 82, which is in the denied band. The system's own containment plan cannot be authorised in production, even by a senior operator. Containing a host that is actively beaconing is exactly what a responder would want to do quickly, so this is a real finding about the thresholds, not about the model.

F. A Confound Found by Reading

The first full RQ1 comparison scored the architecture 31 out of 31. That turned out to be an artifact. In 44 of the first 60 scenarios, and in all 5 executable remediation scenarios, the expected action happened to be listed first among the permitted actions, and the shared proposer picks the first permitted action. Every remediation both arms got right had been won by list order instead of by diagnosing anything. The permitted actions are now sorted so that position carries no signal, and Table VII comes from the corrected corpus. I found this by reading the scenarios, not through a test, which means the same kind of confound may still exist elsewhere.

XV. DISCUSSION

Taken together, the four results support one claim and leave another open. The supported claim is about containment. On the cluster, the policy layer executed no wrong action (RQ1). Across the whole corpus it never let a forbidden action run alone (RQ2). An attacker in full control of the model could not cause a single execution or approval through the agent (RQ3). And real models under injection did not get anything past it either (RQ4). The open claim is about competence. Nothing here shows that the system diagnoses incidents well or resolves them quickly, and RQ1 and RQ2 both show it handing too much to people.

Those two failure modes are not equally bad. A system that hands too much to people is annoying. A system that confidently executes the wrong action in production is dangerous. The runbook in RQ1 got more than half its executed actions wrong. The architecture got none wrong, and its over escalation has a clear, adjustable source: production adds 15 points, most incidents touch more than one entity, and an autonomous threshold of 30 is strict once those penalties apply. The isolate host denial in RQ4 shows the cost concretely. The thresholds and weights are parameters, and the benchmark is the intended tool for tuning them. What the architecture guarantees is that tuning can only move the line between autonomous and human; it cannot let the model move it.

On novelty: none of the individual mechanisms is new. Policy gates, runbooks, safety shields [38] and retrieval with citations [27] all exist. The contribution is the combination, made checkable. In particular, RQ3 gives a bound that does not depend on any model's behaviour, which is the property recent

work on structural injection defences argues for [32], [33]. Pashupatastra shows that bound holding in a deployed incident response system, with the attack surface enumerated rather than sampled. A broader lesson is that most of the safety properties are boring code: exact string matching, a frozen list of identifiers, a verdict with an expiry time, a learning function with one path. Each can be read and tested by someone who does not trust the model, which is the position an operator should be in.

XVI. LIMITATIONS AND THREATS TO VALIDITY

The benchmark is self authored. One person wrote the scenarios, the answer key and the system being scored. Writing scenarios before the resolving logic, including negative and adversarial cases, and reporting failures first all help, but none of them remove the problem. Independent scenario authors are the only real fix.

Diagnosis quality is not measured, and cannot be without the cluster. Every PIB scenario file names its own fault in its title and fault description, so a model shown the file would be paraphrasing an answer it was handed. The only legitimate input is the telemetry an injected fault produces, which exists only while the reference stack runs. The smaller replay scenarios do carry telemetry, but in four of the eight the metric names nearly spell out the root cause, two have no root cause because they are negative cases, and two have answer keys that are not diagnoses, so none can be scored. Root cause accuracy, detection precision and recall, and mean time to recovery remain defined but unreported.

Coverage and records. RQ1 covers 31 of 104 scenarios, and the subset is weighted toward cases whose right answer is to do nothing. The other 73 are excluded by name with a reason: eleven need a simulated adversary, eight a load generator, five would destroy real data, and most of the rest name recovery conditions the stateless stack cannot report. The records behind Table VII are not in the public repository. RQ2 covers all 104, but only at the policy level.

Two of four arms produced no data. The human arm needs operators and the naive model arm needs the cluster. Both refuse to run rather than approximate.

The models are small and the attacks are fixed. RQ4 used three local models of 7 to 8 billion parameters. The deployment's primary model and the production target are different, and larger models may follow injected instructions more readily, because they follow instructions better in general. The eight payloads were written in advance and do not adapt. An adaptive attacker of the kind AgentDojo models [31] has not been tested, although RQ3 bounds what any such attacker could achieve through the chat route.

No outside users. No security practitioner has used the system or reviewed its verdicts. Whether the verdict explanations actually help an operator decide, and whether the approval rate leads to fatigue, are questions only a user study can answer.

Narrow environment and scripted demonstrations. The evaluation covers one reference architecture. The public incidents are scripted and their confidences are chosen, not

measured. The security topology in the public deployment records no dependency edges, so blast radius there has nothing to traverse.

XVII. ETHICAL CONSIDERATIONS

A system that can take security actions is dual use by nature, so the public deployment is permanently in dry run and holds no credentials capable of changing anything. It does not collect personal data from visitors, it does not mint a visitor identifier, and its page view count says plainly that it is unauthenticated and could be inflated. Threat feed entries are shown with defanged indicators and links to their sources. The demonstration incidents use reserved documentation addresses and invented account names. The injection payloads in RQ4 were run only against local models and the in memory demonstration store. Every model turn is stored in a public ledger together with the name of the model that produced it.

XVIII. CONCLUSION AND FUTURE WORK

Pashupatastra is built on the idea that the useful question about an autonomous incident responder is not whether it can restart a service. It is where the boundary on its autonomy sits and whether that boundary is enforced by something other than the model's own judgement. The system puts that boundary in deterministic code: a closed registry of actions, a monotone risk score, explicit tiers and overrides, verdicts that are bound and expire, verification that observes instead of asks, and a memory that keeps its failures. The four evaluations support a claim about containment. No wrong action ran on the cluster, no forbidden action could run alone anywhere in the corpus, and a model under an attacker's full control could not cause a single execution through the agent. They do not yet support a claim about competence.

The work needed to close that gap is specific. Run the PIB stack so that faults produce real telemetry, then measure root cause accuracy for the deployment's actual models with and without the policy layer, at least three runs per scenario. Publish the run records with the paper. Test adaptive injection attacks against frontier models. Recalibrate the thresholds using RQ2 and the isolate host case. And run a small study, five to eight security practitioners working the blue team scenarios and rating the verdict explanations, so that the question of whether this helps a person decide has an answer from someone other than its author. Until then, the right summary is that the architecture is shown to contain a model and is not yet shown to be a good operator.

CODE AND DATA AVAILABILITY

The source code, the PIB scenarios and the benchmark harness are available in the project repository [46]. The scripts and raw JSON records for RQ2 to RQ4, together with the analysis script that recomputes Tables VIII to X, are in the repository's docs/research/experiments folder. The records behind Table VII are not yet published. The public console, including the audit ledger, is available online [7].

REFERENCES

- [1] B. A. Alahmadi, L. Axon, and I. Martinovic, "99% false positives: A qualitative study of SOC analysts' perspectives on security alarms," in *Proc. 31st USENIX Secur. Symp.*, Boston, MA, USA, Aug. 2022. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity22/presentation/alahmadi>
- [2] W. U. Hassan, S. Guo, D. Li, Z. Chen, K. Jee, Z. Li, and A. Bates, "NoDoze: Combatting threat alert fatigue with automated provenance triage," in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, San Diego, CA, USA, 2019, doi: [10.14722/ndss.2019.23349](https://doi.org/10.14722/ndss.2019.23349).
- [3] F. B. Kokulu, A. Soneji, T. Bao, Y. Shoshitaishvili, Z. Zhao, A. Doupé, and G.-J. Ahn, "Matched and mismatched SOC: A qualitative study on security operations center issues," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, London, U.K., 2019, pp. 1955-1970, doi: [10.1145/3319535.3354239](https://doi.org/10.1145/3319535.3354239).
- [4] S. Ghosh, M. Shetty, C. Bansal, and S. Nath, "How to fight production incidents? An empirical study on a large-scale cloud service," in *Proc. 13th Symp. Cloud Comput. (SoCC)*, San Francisco, CA, USA, 2022, pp. 126-141, doi: [10.1145/3542929.3563482](https://doi.org/10.1145/3542929.3563482).
- [5] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1-38, 2023, doi: [10.1145/3571730](https://doi.org/10.1145/3571730).
- [6] L. Bainbridge, "Ironies of automation," *Automatica*, vol. 19, no. 6, pp. 775-779, 1983, doi: [10.1016/0005-1098\(83\)90046-8](https://doi.org/10.1016/0005-1098(83)90046-8).
- [7] P. Raval, "Pashupatastra public console," 2026. [Online]. Available: <https://pashupatastra.vercel.app>
- [8] OpenTelemetry Authors, "OpenTelemetry specification," Cloud Native Computing Foundation. [Online]. Available: <https://opentelemetry.io/docs/specs/otel/>
- [9] T. Ahmed, S. Ghosh, C. Bansal, T. Zimmermann, X. Zhang, and S. Rajmohan, "Recommending root-cause and mitigation steps for cloud incidents using large language models," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, Melbourne, Australia, 2023, pp. 1737-1749, doi: [10.1109/ICSE48619.2023.00149](https://doi.org/10.1109/ICSE48619.2023.00149).
- [10] Y. Chen *et al.*, "Automatic root cause analysis via large language models for cloud incidents," in *Proc. 19th Eur. Conf. Comput. Syst. (EuroSys)*, Athens, Greece, 2024, pp. 674-688, doi: [10.1145/3627703.3629553](https://doi.org/10.1145/3627703.3629553).
- [11] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, Kigali, Rwanda, 2023, arXiv:2210.03629, doi: [10.48550/arXiv.2210.03629](https://doi.org/10.48550/arXiv.2210.03629).
- [12] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop Artif. Intell. Secur. (AISec)*, Copenhagen, Denmark, 2023, pp. 79-90, doi: [10.1145/3605764.3623985](https://doi.org/10.1145/3605764.3623985).
- [13] Pydantic Services, "Pydantic: Data validation using Python type hints." [Online]. Available: <https://docs.pydantic.dev>
- [14] P. Notaro, J. Cardoso, and M. Gerdmt, "A survey of AIOps methods for failure management," *ACM Trans. Intell. Syst. Technol.*, vol. 12, no. 6, pp. 1-45, 2021, doi: [10.1145/3483424](https://doi.org/10.1145/3483424).
- [15] Y. Gan, M. Liang, S. Dev, D. Lo, and C. Delimitrou, "Sage: Practical and scalable ML-driven performance debugging in microservices," in *Proc. 26th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS)*, 2021, pp. 135-151, doi: [10.1145/3445814.3446700](https://doi.org/10.1145/3445814.3446700).
- [16] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Eds., *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol, CA, USA: O'Reilly Media, 2016. [Online]. Available: <https://sre.google/sre-book/table-of-contents/>
- [17] Y. Chen *et al.*, "AIOpsLab: A holistic framework to evaluate AI agents for enabling autonomous clouds," 2025, arXiv:2501.06706, doi: [10.48550/arXiv.2501.06706](https://doi.org/10.48550/arXiv.2501.06706).
- [18] C. Islam, M. A. Babar, and S. Nepal, "A multi-vocal review of security orchestration," *ACM Comput. Surv.*, vol. 52, no. 2, pp. 1-45, 2019, doi: [10.1145/3305268](https://doi.org/10.1145/3305268).
- [19] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symp.*

- Secur. Privacy (SP)*, Oakland, CA, USA, 2010, pp. 305-316, doi: [10.1109/SP.2010.25](https://doi.org/10.1109/SP.2010.25).
- [20] H. Xu *et al.*, “Large language models for cyber security: A systematic literature review,” 2024, arXiv:2405.04760, doi: [10.48550/arXiv.2405.04760](https://doi.org/10.48550/arXiv.2405.04760).
 - [21] A. Nelson, S. Rekhi, M. Souppaya, and K. Scarfone, “Incident response recommendations and considerations for cybersecurity risk management: A CSF 2.0 community profile,” Nat. Inst. Standards Technol., Gaithersburg, MD, USA, NIST SP 800-61r3, Apr. 2025, doi: [10.6028/NIST.SP.800-61r3](https://doi.org/10.6028/NIST.SP.800-61r3).
 - [22] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas, “MITRE ATT&CK: Design and philosophy,” The MITRE Corporation, McLean, VA, USA, Tech. Rep., 2018. [Online]. Available: <https://attack.mitre.org>
 - [23] SigmaHQ, “Sigma: Generic signature format for SIEM systems.” [Online]. Available: <https://github.com/SigmaHQ/sigma>
 - [24] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023, pp. 68539-68551, doi: [10.52202/075280-2997](https://doi.org/10.52202/075280-2997).
 - [25] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, “Identifying the risks of LM agents with an LM-emulated sandbox,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, Vienna, Austria, 2024, arXiv:2309.15817, doi: [10.48550/arXiv.2309.15817](https://doi.org/10.48550/arXiv.2309.15817).
 - [26] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 9459-9474, arXiv:2005.11401, doi: [10.48550/arXiv.2005.11401](https://doi.org/10.48550/arXiv.2005.11401).
 - [27] T. Gao, H. Yen, J. Yu, and D. Chen, “Enabling large language models to generate text with citations,” in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, Singapore, 2023, pp. 6465-6488, doi: [10.18653/v1/2023.emnlp-main.398](https://doi.org/10.18653/v1/2023.emnlp-main.398).
 - [28] N. F. Liu, T. Zhang, and P. Liang, “Evaluating verifiability in generative search engines,” in *Findings Assoc. Comput. Linguistics: EMNLP*, Singapore, 2023, pp. 7001-7025, doi: [10.18653/v1/2023.findings-emnlp.467](https://doi.org/10.18653/v1/2023.findings-emnlp.467).
 - [29] F. Perez and I. Ribeiro, “Ignore previous prompt: Attack techniques for language models,” presented at the NeurIPS ML Safety Workshop, 2022, arXiv:2211.09527, doi: [10.48550/arXiv.2211.09527](https://doi.org/10.48550/arXiv.2211.09527).
 - [30] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, “Formalizing and benchmarking prompt injection attacks and defenses,” in *Proc. 33rd USENIX Secur. Symp.*, Philadelphia, PA, USA, Aug. 2024. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupe>
 - [31] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, “AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, 2024, pp. 82895-82920, doi: [10.52202/079017-2636](https://doi.org/10.52202/079017-2636).
 - [32] E. Debenedetti *et al.*, “Defeating prompt injections by design,” 2025, arXiv:2503.18813, doi: [10.48550/arXiv.2503.18813](https://doi.org/10.48550/arXiv.2503.18813).
 - [33] L. Beurer-Kellner *et al.*, “Design patterns for securing LLM agents against prompt injections,” 2025, arXiv:2506.08837, doi: [10.48550/arXiv.2506.08837](https://doi.org/10.48550/arXiv.2506.08837).
 - [34] OWASP GenAI Security Project, “OWASP top 10 for LLM applications 2025,” OWASP Foundation, 2025. [Online]. Available: <https://genai.owasp.org>
 - [35] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, “A model for types and levels of human interaction with automation,” *IEEE Trans. Syst., Man, Cybern. A, Syst., Humans*, vol. 30, no. 3, pp. 286-297, May 2000, doi: [10.1109/3468.844354](https://doi.org/10.1109/3468.844354).
 - [36] J. García and F. Fernández, “A comprehensive survey on safe reinforcement learning,” *J. Mach. Learn. Res.*, vol. 16, pp. 1437-1480, 2015. [Online]. Available: <https://jmlr.org/papers/v16/garcia15a.html>
 - [37] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, “Concrete problems in AI safety,” 2016, arXiv:1606.06565, doi: [10.48550/arXiv.1606.06565](https://doi.org/10.48550/arXiv.1606.06565).
 - [38] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, “Safe reinforcement learning via shielding,” in *Proc. AAAI Conf. Artif. Intell.*, vol. 32, no. 1, 2018, doi: [10.1609/aaai.v32i1.11797](https://doi.org/10.1609/aaai.v32i1.11797).
 - [39] M. Leucker and C. Schallhart, “A brief account of runtime verification,” *J. Log. Algebr. Program.*, vol. 78, no. 5, pp. 293-303, 2009, doi: [10.1016/j.jlap.2008.08.004](https://doi.org/10.1016/j.jlap.2008.08.004).
 - [40] Open Policy Agent Authors, “Open Policy Agent documentation,” Cloud Native Computing Foundation. [Online]. Available: <https://www.openpolicyagent.org/docs>
 - [41] E. Tabassi, “Artificial intelligence risk management framework (AI RMF 1.0),” Nat. Inst. Standards Technol., Gaithersburg, MD, USA, NIST AI 100-1, Jan. 2023, doi: [10.6028/NIST.AI.100-1](https://doi.org/10.6028/NIST.AI.100-1).
 - [42] R. E. Tarjan, “Efficiency of a good but not linear set union algorithm,” *J. ACM*, vol. 22, no. 2, pp. 215-225, 1975, doi: [10.1145/321879.321884](https://doi.org/10.1145/321879.321884).
 - [43] OpenAI, “gpt-oss-120b and gpt-oss-20b model card,” 2025, arXiv:2508.10925, doi: [10.48550/arXiv.2508.10925](https://doi.org/10.48550/arXiv.2508.10925).
 - [44] Qwen Team, “Qwen2.5 technical report,” 2024, arXiv:2412.15115, doi: [10.48550/arXiv.2412.15115](https://doi.org/10.48550/arXiv.2412.15115).
 - [45] S. Ramírez, “FastAPI.” [Online]. Available: <https://fastapi.tiangolo.com>
 - [46] P. Raval, “Pashupatastra: Policy-bounded autonomous security operations,” software, version 0.1.0, Sep. 2026. [Online]. Available: <https://github.com/preetraval45/PASHUPATASTRA>
 - [47] Cybersecurity and Infrastructure Security Agency, “Known exploited vulnerabilities catalog.” [Online]. Available: <https://www.cisa.gov/known-exploited-vulnerabilities-catalog>
 - [48] abuse.ch, “URLhaus, Feodo Tracker and ThreatFox threat intelligence platforms.” [Online]. Available: <https://abuse.ch>
 - [49] T. Hunt, “Have I Been Pwned.” [Online]. Available: <https://haveibeenpwned.com>
 - [50] “ransomware.live: Ransomware group and victim tracker.” [Online]. Available: <https://www.ransomware.live>